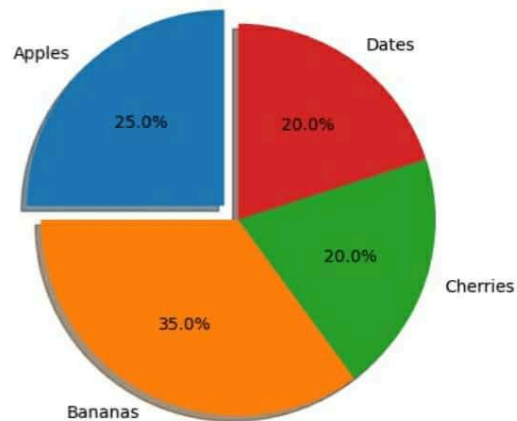
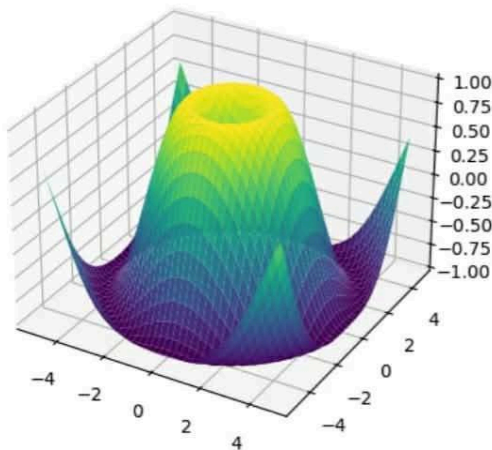


Matplotlib Cheat Sheet !



Prepared With Love By : Saurab Gyawali

[LinkedIn](#) [GitHub](#) [Instagram](#)

1 Data Visualization with Matplotlib

1.1 Importance of visualization in data science

1.2 Plotting basics using Matplotlib

1.2.1 Line

1.2.2 Bar

1.2.3 Scatter,

1.2.4 Histogram

1.3 Customizing plots: Titles, labels, legends, colors, subplots

Prepared With Love By : Saurab Gyawali

1. Data Visualization with Matplotlib

1.1 Importance of Visualization in Data Science

Data visualization is a foundational pillar of data science. It transforms raw numbers into intuitive graphical representations, enabling faster comprehension, pattern recognition, and decision-making.

Large numerical datasets become easy to interpret when converted into visuals.

Visualization allows analysts to instantly notice patterns that raw data cannot clearly reveal.

For Example For A Fruit Vendor His Sales Of Different Fruits Are :

Q. Which One Looks Much More Engaging ?

Option 1 : Numerically

Fruits | Number Of Sales

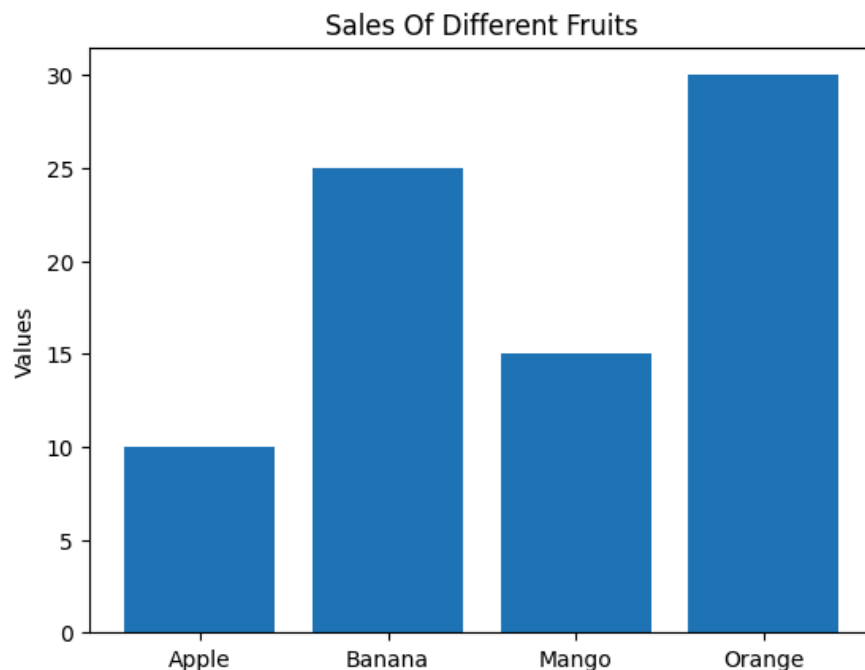
Apple : 10

Banana : 25

Mango : 15

Orange : 30

Option 2 : Graphically



The bar chart quickly shows that Orange has the highest sales, while Apple has the lowest.

1.2 Plotting Basics Using Matplotlib

Matplotlib is Python's primary 2D plotting library. It integrates with NumPy and Pandas and provides fine-grained control over every plot element. Matplotlib is a widely used, open-source plotting library for the Python programming language. It is primarily used for creating static, animated, and interactive visualizations in Python.

Installation (if needed):

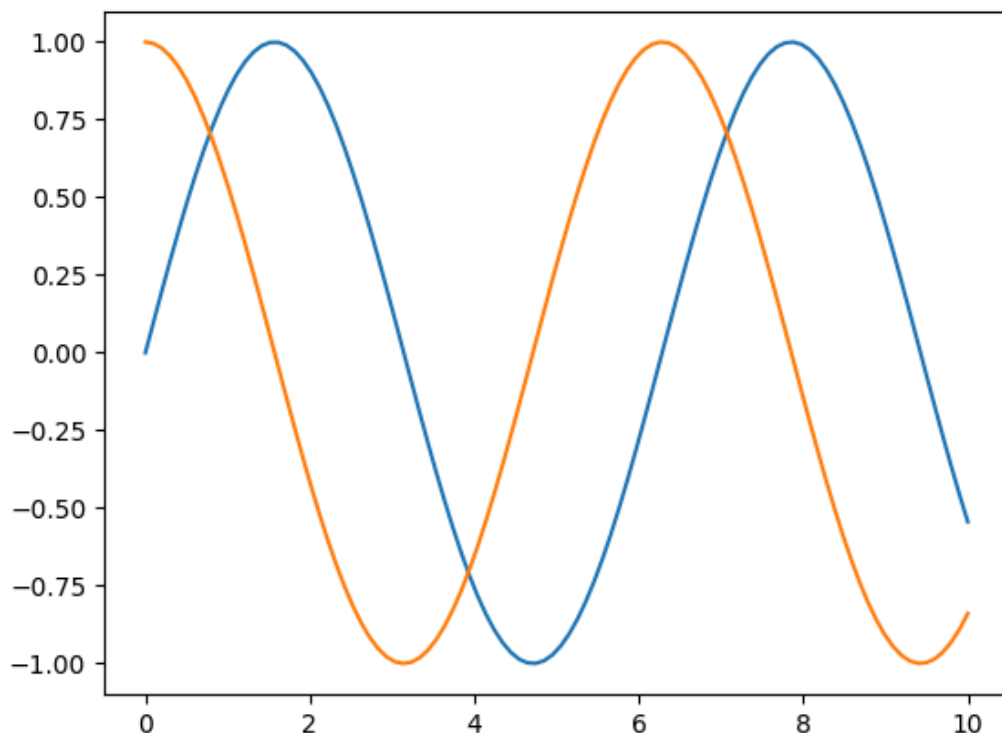
```
import matplotlib.pyplot as plt
```

1.2.1 Line Plot

A line plot visualizes the relationship between two continuous variables by connecting individual data points with straight line segments. It is primarily used to show trends, patterns, or changes over time.

```
import matplotlib.pyplot as plt
import numpy as np

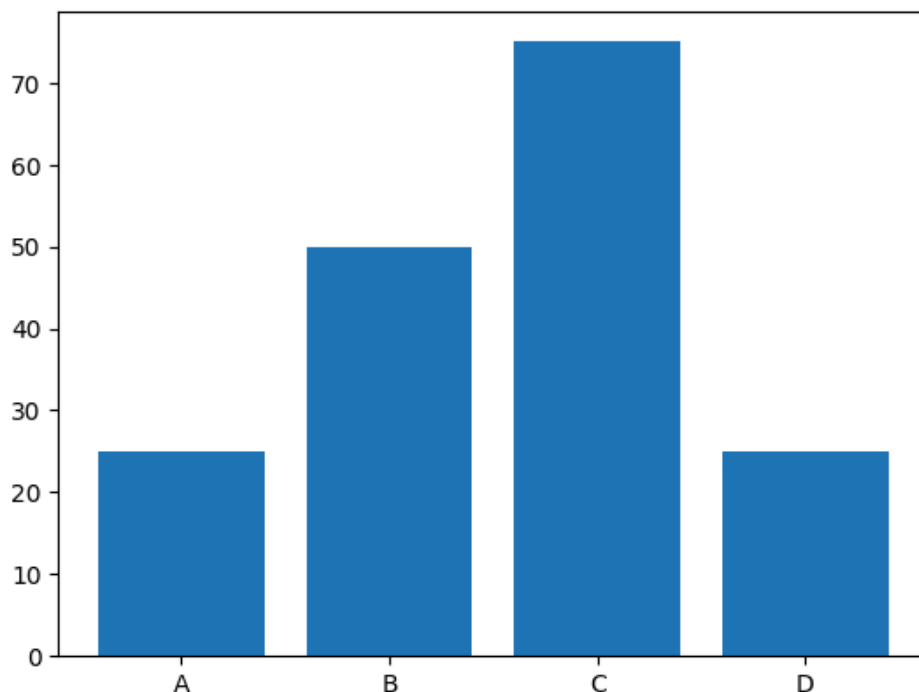
x = np.linspace(0, 10, 100)
plt.plot(x, np.sin(x) )
plt.plot(x, np.cos(x) )
plt.show()
```



1.2.2 Bar Plot

A bar plot represents categorical data using rectangular bars whose heights (or lengths) are proportional to the values they represent. It is ideal for comparing discrete groups, frequencies, or aggregated statistics.

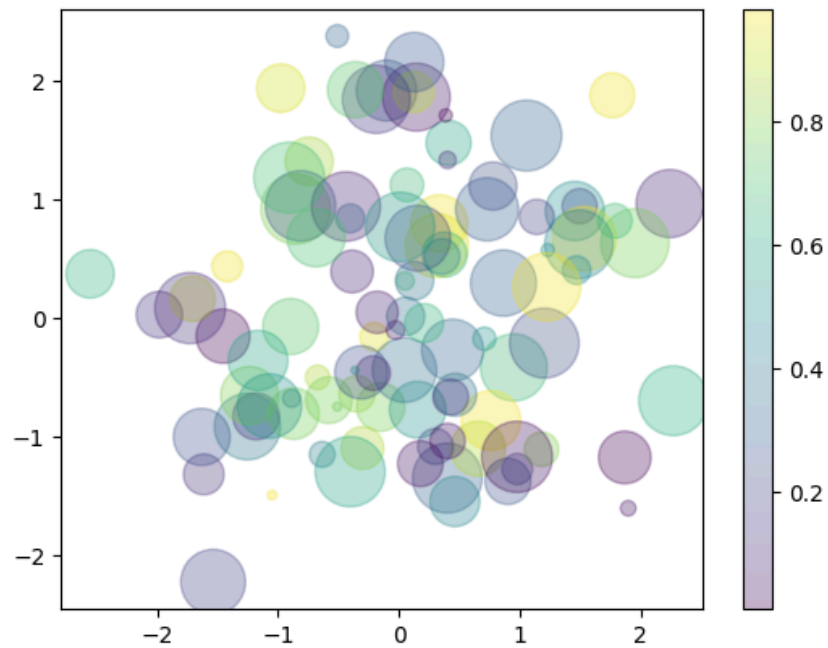
```
categories = ['A', 'B', 'C', 'D']  
values = [25, 50, 75, 25]  
  
plt.bar(categories, values )  
plt.show()
```



1.2.3 Scatter Plot

A scatter plot displays individual data points on a two-dimensional plane to show the relationship or correlation between two continuous variables. It is useful for identifying patterns, clusters, and outliers.

```
import numpy as np  
import matplotlib.pyplot as plt  
rng = np.random.RandomState(0)  
x = rng.randn(100)  
y = rng.randn(100)  
plt.scatter(x, y, c=rng.rand(100), s=1000 * rng.rand(100), alpha=0.3,  
            cmap='viridis')  
plt.colorbar()  
plt.show()
```

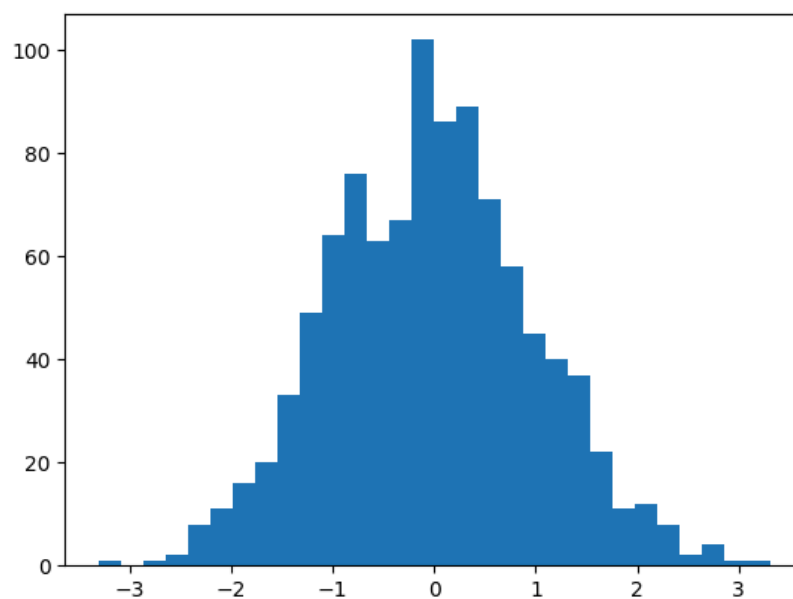


1.2.4 Histogram

A histogram visualizes the distribution of a continuous variable by grouping data into bins and displaying the frequency of observations within each bin. It helps analyze data spread, central tendency, and skewness.

```
import numpy as np
import matplotlib.pyplot as plt

data = np.random.randn(1000)
plt.hist(data, bins=30 )
plt.show()
```



1.3 Customizing Plots

Professional plots require customization for clarity and aesthetics .

Feature	plt.pyplot (Simple)	Axes Object (Subplots/Complex)
Title	plt.title()	ax.set_title()
X-Label	plt.xlabel()	ax.set_xlabel()
Y-Label	plt.ylabel()	ax.set_ylabel()
Plot Data	plt.plot()	ax.plot()
Grid	plt.grid()	ax.grid()

Example: Fully Customized Plot

```
import matplotlib.pyplot as plt
import numpy as np

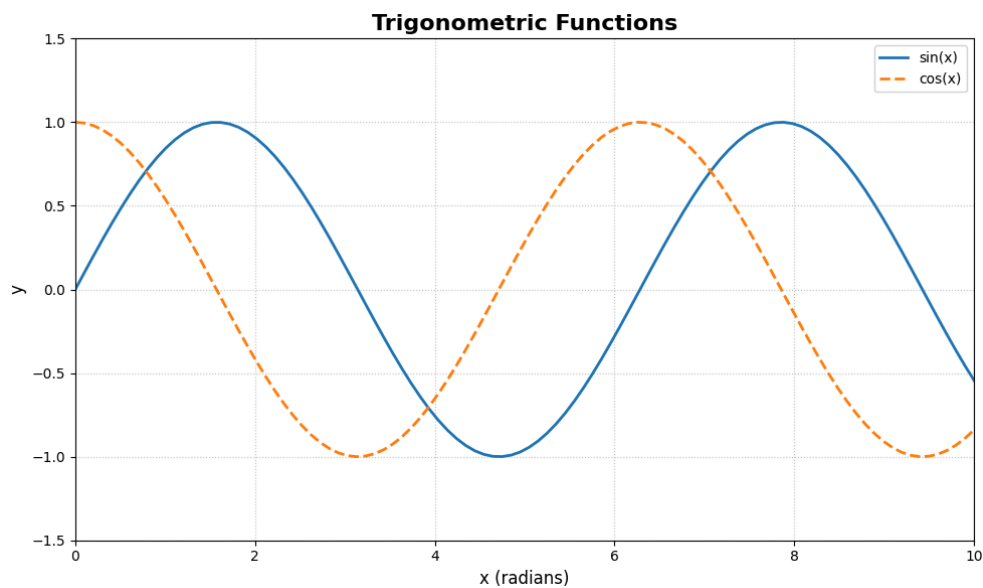
# Generate data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Create figure and axis
plt.figure(figsize=(10, 6))

# Plot both lines
plt.plot(x, y1, label='sin(x)', color='tab:blue', linewidth=2)
plt.plot(x, y2, label='cos(x)', color='tab:orange', linestyle='--',
         linewidth=2)

# Customize
plt.title('Trigonometric Functions', fontsize=16, fontweight='bold')
plt.xlabel('x (radians)', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)
plt.xlim(0, 10)
plt.ylim(-1.5, 1.5)

# Show
plt.tight_layout()
plt.show()
```



1.4 Problems

1.4.1 Theoretical

1. Define data visualization and explain its role in exploratory data analysis (EDA).
2. Why is Matplotlib considered the foundational plotting library in Python?
3. Explain the difference between static, animated, and interactive visualizations.
4. What are the advantages of using Matplotlib over spreadsheet-based plotting tools?

1.4.2 Coding Related

1. Write a Matplotlib code to plot a simple line chart of $x=[1,2,3,4]$ and $y=[2,4,6,8]$.
2. Generate a bar chart showing sales of Apple, Banana, Mango, Orange with values $[10,25,15,30]$.
3. Write code to create a scatter plot of random 50 points with color customization.
4. Create a histogram of 1000 normally distributed random values.
5. Plot a line and scatter plot on the same figure.

Drop Your *Gmail* In The Comment , I Will Send You All Materials !!

Found this helpful ?

**Save & Follow for
more content**



@saurab_gyawali



Like



Comment



Share



Save